

Basic GPU Kernels — Placement and Interview Guide

This guide covers eight foundational data-parallel kernels. Examples use CUDA-like pseudocode, but the reasoning applies to CUDA, HIP, OpenCL, Metal, and other SIMT/SIMD systems. In the pseudocode, one logical GPU thread handles one output element unless stated otherwise.

Vector Addition

1. Overview

Definition. Vector addition computes $C[i] = A[i] + B[i]$ for every index i . The elements may be integers, floating-point values, or small vector types.

It matters because it is the smallest useful example of GPU data parallelism: every output is independent, so thousands of threads can work simultaneously. Real systems use the same pattern in scientific arrays, neural-network residual connections, signal processing, finance, and simulation state updates. Interviewers ask it to test whether you understand thread indexing, bounds checks, grid sizing, memory coalescing, launch overhead, and why a kernel can be bandwidth-bound even when it has massive parallelism.

2. Core Idea

Think of two rows of boxes. A worker assigned box i reads box i from each row, adds the values, and writes one result. Workers never communicate.

For $A=[2,4,6,8]$ and $B=[1,3,5,7]$, four logical threads produce $C=[3,7,11,15]$.

```
__global__ void add(const float* a, const float* b, float* c, size_t n) {
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}

// host-side launch
int threads = 256;
int blocks = (n + threads - 1) / threads;
add<<<blocks, threads>>>(a, b, c, n);
```

Step by step: the runtime creates a grid; each thread computes a globally unique i ; valid threads load two values; the ALU adds them; the result is stored. The ceiling division launches enough threads for a non-multiple of the block size, while the guard makes the extra threads harmless.

3. Important Subtopics

Subtopic	Meaning and importance	Example	Interview angle
Global indexing	Converts block-local IDs into an array index.	Block 3, 256 threads, local ID 5 gives 773.	Derive the formula and explain the guard.
Grid/block sizing	Chooses execution shape; 128–256 threads is a common starting point, not a law.	<code>ceil(n/256)</code> blocks.	Occupancy is not automatically performance.
Coalescing	Adjacent lanes should access adjacent addresses so requests combine into few memory transactions.	Lane <code>k</code> reads <code>a[base+k]</code> .	Why strided layouts waste bandwidth.
Arithmetic intensity	Useful operations per byte moved. Vector add does one add while reading two values and writing one.	FP32: roughly 1 FLOP / 12 bytes.	Classify it as memory-bandwidth-bound.
Aliasing/in-place use	<code>c</code> may equal <code>a</code> or <code>b</code> safely because each index is read before the same index is written and there are no cross-index dependencies.	<code>add(a,b,a,n)</code> .	State conditions under which aliasing is safe.
Transfers and launch cost	Host-device copies and launch latency can exceed kernel time for small arrays.	Adding 100 floats on a GPU is often slower than CPU code.	Discuss end-to-end time, not kernel time only.
Grid-stride loop	Lets a fixed grid process arrays larger than the grid and can reuse threads.	<code>for(i=tid;i<n;i+=gridDim.x*blockDim.x)</code> .	Useful for large inputs and persistent launch sizes.

4. Real-World Example

A neural network residual block computes `output = learned_transform + input`. Both tensors already reside on the GPU; a vector-add-like kernel combines them. Production libraries often fuse this add with

activation, bias, or normalization to avoid another full read/write pass. The important practical lesson is that keeping data resident and reducing memory traffic usually matters more than optimizing the single addition.

5. Diagrams / Mental Models

Grid

Block 0: T0 T1 T2 T3 -> C[0..3]

Block 1: T0 T1 T2 T3 -> C[4..7]

A: [a0 a1 a2 a3 a4 a5 a6 a7]

B: [b0 b1 b2 b3 b4 b5 b6 b7]

+ + + + + + + +

C: [c0 c1 c2 c3 c4 c5 c6 c7]

Property	Vector addition
Work	$O(n)$ additions
Parallel depth	$O(1)$ with enough processors
Global traffic (FP32)	about $12n$ bytes
Inter-thread communication	None
Typical bottleneck	Memory bandwidth / overhead

6. Common Interview Questions

Question	Clear answer	Interviewer expects	Common mistake
1. How does a thread find its element?	$i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$.	Grid hierarchy.	Using only threadIdx.x .
2. Why check $i < n$?	Ceiling division may create extra threads.	Safe arbitrary sizes.	Assuming n is divisible by 256.
3. Why is it bandwidth-bound?	One cheap add requires two loads and one store.	Arithmetic intensity reasoning.	Saying more cores always make it compute-bound.
4. What is coalescing?	A warp's adjacent lanes access adjacent aligned locations, allowing combined transactions.	Access pattern, not merely contiguous allocation.	Claiming shared memory is required.
5. Can it run in place?	Yes, if each output uses only same-index inputs and races with no other operation.	Aliasing reasoning.	Giving an unconditional yes

Question	Clear answer	Interviewer expects	Common mistake
			for arbitrary stencils.
6. Why might CPU be faster?	Small n , launch latency, copies, synchronization, or data already on CPU.	End-to-end cost.	Comparing only GPU arithmetic throughput.
7. How many blocks?	$(n+T-1)/T$ for T threads per block.	Ceiling division.	Integer truncation with n/T .
8. Should inputs use shared memory?	Usually no; each value is used once, so staging adds work without reuse.	Reuse criterion.	Using shared memory because it is faster in isolation.
9. How do you time it?	Warm up, use GPU events around asynchronous work, repeat, and synchronize appropriately.	Asynchronous execution awareness.	CPU timer without synchronization.
10. How can it be optimized?	Coalesced accesses, resident data, vectorized aligned loads when justified, and fusion with neighboring operations.	Memory traffic reduction.	Micro-optimizing the add instruction.

7. Deep-Dive Questions

- What is the roofline interpretation?** Performance is limited near $\text{memory_bandwidth} \times \text{arithmetic_intensity}$; with about 1/12 FLOP/byte for FP32, the bandwidth roof is reached long before peak FLOP/s.
- Would float4 loads help?** They can reduce instruction count when addresses and length are suitably aligned, but they do not reduce required bytes and may not improve an already saturated kernel. Handle the tail separately.
- What changes for multidimensional tensors?** Either flatten contiguous storage or compute multidimensional coordinates. Strides and broadcasting may make access non-contiguous and require careful mapping.
- How does unified memory affect it?** First-touch page migration can dominate the kernel. Prefetching or establishing residency makes measurements representative.
- Why fuse operations?** $D = \text{relu}(A+B)$ in one kernel avoids writing and rereading the intermediate C , trading modularity for lower global-memory traffic and fewer launches.

8. Comparison Tables

CPU loop	GPU kernel
Low setup cost	Launch and possible transfer cost
Few powerful cores	Many lightweight threads
Good for small/nonresident data	Good for large resident arrays

CPU loop	GPU kernel
Cache-sensitive sequential loop	Throughput from coalesced parallel access
Direct indexing	Grid-stride loop
One element per launched thread	Several elements per thread possible
Simplest for ordinary sizes	Fixed grid can cover huge arrays
More blocks as <code>n</code> grows	May improve reuse/control launch size

9. Common Mistakes

- Omitting the bounds check or calculating blocks with floor division.
- Measuring copies and kernel inconsistently, or forgetting asynchronous execution.
- Choosing a block size solely for maximum occupancy.
- Using shared memory despite zero data reuse.
- Ignoring integer overflow in index arithmetic for very large arrays.
- Expecting floating-point results to obey real-number associativity in later fused computations.

10. Edge Cases / Special Cases

- `n=0`: avoid an invalid zero-block launch or simply skip it on the host.
- `n` not divisible by the block size: extra threads must return/do nothing.
- Huge `n`: use `size_t`/64-bit indexing and possibly a grid-stride loop.
- Misaligned pointers: scalar access remains correct; vectorized access needs alignment handling.
- Overlapping buffers with different offsets can race even though exact in-place aliasing is safe.
- Integer addition can overflow; signed C++ overflow semantics and GPU instructions must match requirements.

11. How to Explain in Interview

“Vector addition is an embarrassingly parallel, memory-bound kernel. I launch one thread per element, compute a global index, guard it against `n`, read corresponding values from two contiguous arrays, add them, and write one result. Adjacent threads give coalesced accesses. For small inputs launch and transfer overhead dominate; for large resident inputs performance is mainly limited by global-memory bandwidth, so fusion is the strongest optimization.”

12. Quick Revision Notes

- Formula: `C[i]=A[i]+B[i]`; no cross-thread dependency.
- Index: `blockIdx.x*blockDim.x+threadIdx.x`; blocks use ceiling division.
- FP32 traffic: 8 bytes read + 4 bytes written for one FLOP.
- Coalesced global memory is essential; shared memory normally adds no value.
- Trap: high parallelism does not mean compute-bound.

13. Practice Tasks

1. Implement CPU and GPU versions; test `n={0,1,255,256,257,10^7}`.

2. Add a grid-stride loop and compare several grid sizes.
3. Measure kernel-only time versus allocation + transfer + kernel time.
4. Change the kernel to $C = \alpha A + B$ and calculate arithmetic intensity.
5. Implement safe exact in-place variants and deliberately test unsafe offset overlap.

14. Final Cheat Sheet

Item	Recall
Core definition	Element-wise $C[i] = A[i] + B[i]$
Why it matters	Canonical independent data-parallel, bandwidth-bound kernel
Most asked	Indexing, bounds, coalescing, block count, CPU-vs-GPU crossover
Common comparison	Direct indexing vs grid-stride; CPU latency vs GPU throughput
One-line answer	One guarded thread per element with coalesced access; bandwidth, not ALU rate, usually limits it.

Vector Scaling

1. Overview

Definition. Vector scaling multiplies every vector element by a scalar: $Y[i] = \alpha * X[i]$. The in-place BLAS operation $x = \alpha * x$ is commonly called **SCAL**.

Scaling appears in normalization, learning-rate updates, graphics transformations, signal gain, numerical solvers, and unit conversion. It matters as a simple map operation and as a building block in operations such as AXPY ($Y = \alpha * X + Y$). Interviewers use it to probe memory bandwidth, scalar argument handling, in-place safety, numerical behavior, and kernel fusion.

2. Core Idea

Imagine a factory conveyor where every price tag must be converted from dollars to rupees using the same exchange rate. Each worker reads one tag, applies the shared multiplier, and replaces or writes it. No worker needs another worker's value.

For $X = [1, -2, 3]$ and $\alpha = 0.5$, threads compute $Y = [0.5, -1, 1.5]$.

```
__global__ void scale(const float* x, float* y, float alpha, size_t n) {
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = alpha * x[i];
}
```

α is identical for all threads and is typically passed as a kernel argument; hardware can broadcast a uniform value efficiently. Each thread performs one load, one multiplication, and one store.

3. Important Subtopics

Subtopic	Meaning / why it matters	Example	Interview angle
Out-of-place vs in-place	Write Y separately or overwrite X ; exact in-place is safe.	<code>x[i]*=alpha.</code>	Memory footprint and alias safety.
Uniform scalar	All lanes use the same alpha ; no per-element load is necessary.	Kernel argument/constant path.	Do not allocate an alpha array.
Arithmetic intensity	One multiply for a load and store: FP32 out-of-place is about 1 FLOP/8 bytes.	Bandwidth-bound.	Roofline classification.
Special scalars	0, 1, -1 , NaN, and infinity affect semantics and possible fast paths.	alpha=1 makes out-of-place scaling a copy.	Avoid optimizations that change NaN/signed-zero behavior unless allowed.
Precision	Half-precision storage may multiply in FP16 or promote to FP32.	ML mixed precision.	Accuracy/performance tradeoff.
Fusion	Scaling can be folded into a consuming kernel.	Matrix output alpha*AB+beta*C.	Save launch and memory traffic.
Vectorization	Aligned packed operations reduce instruction overhead, not byte traffic.	float4.	Tail and alignment correctness.

4. Real-World Example

In gradient descent, a gradient tensor may be scaled by the learning rate before being subtracted from model weights. A separate scale kernel is correct, but production optimizers normally fuse scaling, momentum, weight decay, and parameter update. Fusion reads each tensor fewer times and avoids temporary storage while preserving the conceptual scaling step.

5. Diagrams / Mental Models

```

shared scalar alpha
      /   |   \
X: [ x0  x1  x2  x3 ]
    *   *   *   *
  
```

```
Y: [a*x0 a*x1 a*x2 a*x3]
    T0  T1  T2  T3
```

Variant	Reads	Writes	Independence
Out-of-place $y=a*x$	n	n	Complete
In-place $x*=a$	n	n	Complete
AXPY $y=a*x+y$	$2n$	n	Complete per index

6. Common Interview Questions

Question	Answer	Expected key point	Common mistake
1. What is vector scaling?	Multiply every element by one scalar.	Element-wise map.	Confusing it with vector dot product.
2. Is it parallel?	Yes; outputs have no cross-index dependency.	One thread per element.	Adding synchronization.
3. Is in-place safe?	Exact aliasing is safe because each thread reads/writes only its own index.	Dependency analysis.	Assuming all overlap is safe.
4. What limits speed?	Usually global-memory bandwidth and overhead.	Low arithmetic intensity.	Peak multiplication throughput.
5. Where is α stored?	Passed by value as a uniform kernel argument; compiler/hardware uses an efficient uniform/constant path.	Broadcast behavior.	A global array with repeated scalar.
6. Optimize $\alpha==1$?	In-place can skip the kernel; out-of-place still needs a copy. Consider required IEEE semantics and dispatch overhead.	Semantic distinction.	Returning early for out-of-place and leaving Y unset.
7. What is AXPY?	$Y=\alpha*X+Y$, a scaled vector addition BLAS primitive.	Relationship to scaling/addition.	Calling it a dot product.
8. Why fuse scaling?	Avoid temporary reads/writes and launch overhead.	Memory traffic.	Claiming it increases mathematical parallelism.
9. How handle FP16?	Often accumulate/multiply in higher precision when accuracy requires it; packed half operations can improve throughput.	Mixed precision.	Assuming FP16 always yields identical results.
10. How choose block	Benchmark sensible multiples of warp size such as 128/256; ensure enough	Empirical tuning.	Treating 1024 as always fastest.

Question	Answer	Expected key point	Common mistake
size?	blocks to saturate memory.		

7. Deep-Dive Questions

1. **Can multiplication by zero be replaced by filling zeros?** Not under strict IEEE semantics: $0 * \text{NaN}$ is NaN and $0 * \text{Inf}$ is NaN. It is valid only under relaxed semantics or known-finite input.
2. **How does scaling participate in BLAS GEMM?** GEMM commonly computes $C = \alpha * A * B + \beta * C$, folding scale factors into tiled matrix multiplication so no standalone scaling pass is necessary.
3. **Does in-place reduce traffic?** It removes a second allocation, but still performs a load and store per element; it does not magically update registers without global traffic.
4. **When can memory alignment matter?** It affects transaction efficiency and packed loads. A scalar coalesced kernel handles arbitrary normal alignment; explicit vector loads require stricter alignment.
5. **How would broadcasting generalize?** Per-channel scaling maps an index to a channel-specific coefficient. That adds indexing and coefficient reuse, often benefiting from cache/constant memory.

8. Comparison Tables

Scaling	Vector addition	AXPY
$y = a * x$	$z = x + y$	$y = a * x + y$
1 multiply	1 add	1 multiply + 1 add
1 input vector	2 input vectors	2 input vectors
Often in-place	Output may alias an input	Commonly updates y
In-place	Out-of-place	
Lower allocation requirement	Preserves input	
Exact alias is safe	Clear ownership and pipeline semantics	
Cannot retain original values	May cost extra memory	

9. Common Mistakes

- Confusing scalar-vector multiplication with a dot product.
- Reading the scalar from a full-length array.
- Returning early for $\alpha == 1$ in an out-of-place operation.
- Believing in-place eliminates global-memory traffic.
- Ignoring overflow, underflow, NaNs, infinities, and signed zero.
- Launching a standalone scale kernel when the next operation can naturally absorb it.

10. Edge Cases / Special Cases

- Empty vectors require no launch.
- $\alpha = 1$: no-op only for in-place; out-of-place is a copy.
- $\alpha = 0$: result is not necessarily all positive zero under strict floating-point rules.

- Integer vectors can overflow or truncate when the scalar is non-integral.
- Very small/subnormal floats may flush to zero depending on mode/hardware.
- Offset overlap between **X** and **Y** can create races; exact alias or disjoint buffers are the simple safe cases.

11. How to Explain in Interview

“Vector scaling applies the same scalar to every element, so it maps naturally to one guarded GPU thread per index. The scalar is uniform and efficiently broadcast, accesses are contiguous, and exact in-place operation is safe. Since the kernel performs one multiply for a load and store, it is normally bandwidth-bound; in real pipelines I first look for an opportunity to fuse it with the consumer.”

12. Quick Revision Notes

- $Y[i] = \alpha * X[i]$; BLAS name: SCAL. AXPY adds **Y**.
- Independent elements; no shared memory or synchronization needed.
- Uniform scalar, coalesced vector access, low arithmetic intensity.
- Exact in-place is safe; arbitrary overlapping ranges are not.
- Trap: special-case shortcuts may alter IEEE behavior.

13. Practice Tasks

1. Implement out-of-place and in-place FP32 scaling.
2. Test sizes around block boundaries and scalars $\{0, 1, -1, \text{NaN}, \text{Inf}\}$.
3. Implement AXPY and compare separate scale+add against one fused kernel.
4. Measure effective bandwidth as $(\text{bytes read+written})/\text{time}$.
5. Add per-channel scaling to an **NCHW** tensor and examine coalescing.

14. Final Cheat Sheet

Item	Recall
Core	$Y[i] = \alpha * X[i]$
Why	Basic map used by BLAS, ML, graphics, and solvers
Most asked	In-place safety, bandwidth bound, fusion, special scalars
Compare	SCAL vs add vs AXPY; in-place vs out-of-place
One line	A uniform, independent multiply per element whose real optimization target is memory traffic.

Matrix Addition

1. Overview

Definition. Matrix addition combines equal-shaped matrices element by element: $C[r, c] = A[r, c] + B[r, c]$. It is not matrix multiplication; no row-column reduction occurs.

It is used in tensor residuals, image blending, numerical methods, batched state updates, and graph/analytics pipelines. It matters because a two-dimensional problem must be mapped correctly onto a linear memory layout. Interviewers ask about 2D grids, row-major indexing, pitch/leading dimensions, coalescing, shape compatibility, broadcasting, and bandwidth limits.

2. Core Idea

Overlay two spreadsheets of the same size. Each worker owns one cell and adds the two values at that coordinate.

```
[1 2 3]   [10 20 30]   [11 22 33]
[4 5 6] + [40 50 60] = [44 55 66]
```

```
__global__ void matrix_add(const float* a, const float* b, float* c,
                           int rows, int cols) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row < rows && col < cols) {
        size_t i = (size_t)row * cols + col;
        c[i] = a[i] + b[i];
    }
}
```

A 2D block such as `dim3(32,8)` makes adjacent `x` lanes traverse columns, which are adjacent in row-major memory. The grid dimensions use ceiling division independently for rows and columns.

3. Important Subtopics

Subtopic	Meaning / importance	Example	Interview angle
Shape compatibility	Plain addition requires equal row/column counts.	$M \times N + M \times N$.	Contrast with broadcasting.
Row-major indexing	Logical (r, c) maps to $r * ld + c$; <code>ld</code> may exceed logical columns.	Pitched allocation.	Use leading dimension, not blindly <code>cols</code> .
2D launch mapping	$x \rightarrow \text{columns}$, $y \rightarrow \text{rows}$ naturally matches row-major storage.	<code>block=(32,8)</code> .	Derive grid and bounds.
Coalescing	Warp lanes should normally vary across columns.	Consecutive <code>col</code> .	Swapping row/column mapping causes strided loads.
Flattening	Treat a contiguous matrix as a vector of <code>rows*cols</code> elements.	Reuse vector-add kernel.	Often the simplest correct implementation.
Broadcasting	Smaller dimensions of size one may be reused, requiring index	Add one bias per column.	More complex than equal-shape addition.

Subtopic	Meaning / importance	Example	Interview angle
	mapping.		
Pitched memory	Rows can have padding for alignment.	Byte pitch from allocator.	Correct address uses pitch/leading dimension.

4. Real-World Example

In a transformer residual connection, two `batch × sequence × hidden` tensors are added. Although logically multidimensional, contiguous tensors can be flattened into one vector addition. If bias and activation follow, a framework may fuse all three operations. The logical matrix/tensor shape is still needed for broadcasting and validation, but physical contiguity allows the simplest launch.

5. Diagrams / Mental Models

```

Logical matrix          Row-major memory
(0,0) (0,1) (0,2)  -->  [0,0][0,1][0,2][1,0][1,1][1,2]
(1,0) (1,1) (1,2)

Warp direction:  T0 -> col 0, T1 -> col 1, T2 -> col 2 ...
                  adjacent lanes, adjacent addresses

```

Mapping	Correct?	Memory behavior
Flattened 1D	Yes for contiguous equal shapes	Naturally coalesced
2D, <code>x=column</code>	Yes	Naturally coalesced row-wise
2D, <code>x=row</code>	Correct with right formula	Usually strided and inefficient

6. Common Interview Questions

Question	Answer	Expected	Common mistake
1. What shapes can be added?	Equal shapes for plain addition; broadcast rules may permit size-1 axes.	Validation.	Applying matrix-multiplication compatibility.
2. How map <code>(r,c)</code> to memory?	Row-major: <code>r*leading_dimension+c</code> .	Layout awareness.	Always using <code>r*rows+c</code> .
3. Why map <code>threadIdx.x</code> to columns?	Adjacent lanes then access adjacent row-major elements.	Coalescing.	Mapping x to rows without considering stride.
4. Can the matrix be flattened?	Yes if participating regions are contiguous and shapes/layouts align.	Simpler equivalent problem.	Flattening pitched/noncontiguous views incorrectly.

Question	Answer	Expected	Common mistake
5. Is shared memory useful?	Normally no; each element is used once.	Reuse-based reasoning.	Tiling every matrix operation.
6. Complexity?	$O(MN)$ work and memory, constant parallel depth ideally.	Work vs parallel time.	Saying $O(M+N)$.
7. What is pitch?	Physical byte distance between row starts, possibly greater than row width.	Padding/alignment.	Indexing pitched data as tightly packed.
8. Is it bandwidth-bound?	Yes: one add for two loads and one store.	Arithmetic intensity.	Confusing it with compute-heavy GEMM.
9. How launch?	2D block/grid with ceiling division, or 1D over total elements.	Both valid mappings.	Missing either row or column bound.
10. Can C alias an input?	Exact element-wise alias is safe; arbitrary overlapping views may race.	Dependency/alias reasoning.	Blanket claim that all overlap works.

7. Deep-Dive Questions

- How do leading dimensions support submatrices?** A view can begin at an offset while row starts remain $1d$ elements apart. Index with the parent stride and view offset, not the view width.
- How is column-vector broadcasting implemented?** For $C[r, c] = A[r, c] + v[r]$, all columns in a row reuse $v[r]$; caching helps naturally, but index mapping must prevent an unnecessary replicated vector.
- What if matrices have different layouts?** The kernel needs separate strides for each operand/output. A transpose-like access may be uncoalesced; layout conversion or a specialized fused operation can help.
- Why is a 1D kernel sometimes better?** It has less index arithmetic and simpler launch logic when storage is contiguous; a 2D mapping mainly improves clarity or handles strides/shapes.
- How would you validate performance?** Report effective bandwidth using total logical bytes, confirm coalescing with a profiler, and separate allocation/copy/launch from steady-state kernel timing.

8. Comparison Tables

Matrix addition	Matrix multiplication
Equal output coordinate combines two values	Each output reduces products across an inner dimension
$O(MN)$ work	$O(MNK)$ work
No inter-thread cooperation required	Tiling and reuse are central
Memory-bandwidth-bound	Often compute-bound when well tiled

Contiguous	Pitched/strided
$\text{index} = r * \text{cols} + c$	$\text{index} = r * \text{ld} + c$ or byte-pitch address
Safe to flatten	Flattening may include padding/wrong elements
Simplest kernel	Flexible views/alignment

9. Common Mistakes

- Using multiplication's shape rule instead of equal-shape addition rules.
- Mixing rows and columns in the linear index.
- Launching x across rows in row-major storage and losing coalescing.
- Ignoring separate input/output strides and pitch.
- Using shared-memory tiling with no reuse.
- Flattening a noncontiguous slice.

10. Edge Cases / Special Cases

- Zero rows or columns: skip launch.
- Rectangular and non-block-multiple shapes need independent bounds checks.
- Pitched allocations and submatrix views require correct leading dimensions.
- Broadcasting size-one dimensions changes address calculation.
- Mixed types may require promotion and defined rounding/overflow behavior.
- Exact in-place output is safe; partially overlapping matrix views are not necessarily safe.

11. How to Explain in Interview

"Matrix addition is element-wise, so every output cell is independent. For row-major data I either flatten a contiguous matrix or launch a 2D grid with x mapped to columns, compute (row, col) , check both bounds, and address each operand using its leading dimension. Adjacent lanes then access adjacent columns. Like vector addition, it is bandwidth-bound and normally gains nothing from shared memory unless another fused operation reuses data."

12. Quick Revision Notes

- Plain rule: equal shapes; result has same shape.
- Row-major offset: $\text{row} * \text{ld} + \text{col}$; $\text{ld} == \text{cols}$ only when tightly packed.
- Flatten contiguous inputs; use 2D/strides for views.
- Map warp lanes along contiguous dimension.
- Trap: matrix addition does not need GEMM-style tiling.

13. Practice Tasks

1. Implement both flattened and 2D versions and verify rectangular sizes.
2. Add distinct leading dimensions for all three matrices.
3. Implement row-bias and column-bias broadcasting.
4. Compare correct and transposed thread mappings in a memory profiler.
5. Fuse matrix addition with ReLU and calculate saved bytes.

14. Final Cheat Sheet

Item	Recall
Core	$C[r, c] = A[r, c] + B[r, c]$ for compatible shapes
Why	Common tensor/residual operation and layout exercise
Most asked	2D indexing, coalescing, pitch, flattening, bandwidth
Compare	Addition vs multiplication; contiguous vs pitched
One line	Independent cell-wise addition, mapped across contiguous columns and indexed with the true leading dimensions.

Matrix Multiplication

1. Overview

Definition. For A of shape $M \times K$ and B of shape $K \times N$, matrix multiplication produces C of shape $M \times N$ where $C[i, j] = \sum_{k=0}^{K-1} A[i, k] B[k, j]$.

GEMM is central to neural networks, graphics, scientific simulation, recommendation systems, and data analytics. It matters because it combines massive parallelism with reusable data and exposes the GPU memory hierarchy. Interviewers ask it to test shape reasoning, dot products, tiling, shared memory, synchronization, coalescing, arithmetic intensity, numerical accumulation, and performance tradeoffs.

2. Core Idea

Each output cell is the dot product of one row of A and one column of B .

$$\begin{array}{ccccc}
 A \ (2 \times 3) & & B \ (3 \times 2) & & C \ (2 \times 2) \\
 \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} & \times & \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} & = & \begin{bmatrix} 1*7+2*9+3*11 & 1*8+2*10+3*12 \\ 4*7+5*9+6*11 & 4*8+5*10+6*12 \end{bmatrix}
 \end{array}$$

A naive GPU assigns one thread to each $C[\text{row}, \text{col}]$; that thread loops over K . It is correct but repeatedly fetches values neighboring threads also need.

```
float sum = 0;
for (int k=0; k<K; ++k) sum += A[row*K+k] * B[k*N+col];
C[row*N+col] = sum;
```

A tiled kernel has a block cooperatively load a tile of A and B into shared memory, synchronize, reuse them for many multiply-adds, synchronize before overwriting the tiles, and repeat across K . This converts expensive

global loads into reused on-chip accesses.

3. Important Subtopics

Subtopic	Meaning / importance	Example	Interview angle
Shape rule	Inner dimensions match; output uses outer dimensions.	$(M \times K) (K \times N) \rightarrow M \times N$.	First correctness check.
Naive mapping	One thread computes one dot product.	2D output grid.	Easy baseline, redundant loads.
Tiling	Break K and output into blocks loaded cooperatively.	16×16 or 32×32 conceptual tiles.	Explain reuse, not memorize tile size.
Shared memory	On-chip storage shared by a block.	Cache A/B tiles.	Synchronization and bank conflicts.
Coalescing	Cooperative loads and stores should use adjacent addresses.	Threads load row-wise.	B's logical column access needs careful mapping.
Register blocking	One thread computes multiple output values, reusing operands in registers.	2×2 microtile.	More reuse vs registers/occupancy.
Arithmetic intensity	Tiling increases FLOPs per global byte.	Each loaded tile value serves multiple FMAs.	Why optimized GEMM can be compute-bound.
Precision	Products and sums may use different formats.	FP16 inputs, FP32 accumulation.	Error and tensor-core use.
Edge tiles	Dimensions need not be tile multiples.	Zero-fill invalid loads; guard stores.	Avoid early return before barriers.

4. Real-World Example

A fully connected neural-network layer computes $Y=XW+b$: a large GEMM followed by bias and activation. Vendor libraries select kernels based on shapes, types, alignment, and hardware, often using tensor cores and fusing epilogues. An application should normally call cuBLAS/rocBLAS rather than maintain its own GEMM; the tiled kernel remains essential interview material because it demonstrates GPU locality and cooperation.

5. Diagrams / Mental Models

B tile

[#####]

[#####]

|

A tile [#####] --+--> C output tile

[#####] each A value reused across columns

each B value reused across rows

For tile phase p:

```
global load -> __syncthreads -> T multiply-accumulates
            -> __syncthreads -> next phase
```

Version	Global-memory reuse	Complexity	Likely limit
Naive	Poor	Simple	Memory bandwidth
Shared-memory tiled	Good within block	Moderate	Compute/on-chip resources
Vendor GEMM	Multilevel, architecture-tuned	Hidden behind API	Near hardware roof

6. Common Interview Questions

Question	Answer	Expected	Common mistake
1. What shape is the result?	$(M \times K)(K \times N)$ produces $M \times N$.	Inner/outer dimensions.	Requiring same matrix shapes.
2. What does one output contain?	Dot product of row i of A and column j of B.	Reduction over K.	Element-wise multiplication.
3. Naive complexity?	$O(MNK)$ operations; MN threads may each loop K.	Work and mapping.	Calling it $O(MN)$.
4. Why tile?	Reuse values from expensive global memory across multiple output calculations.	Data reuse.	Saying shared memory changes the math.
5. Why two barriers per phase?	One ensures tile loads finish; the other ensures all reads finish before tile storage is overwritten.	Race prevention.	Omitting the second barrier.
6. Handle nonmultiple sizes?	Guard loads, place zeros for invalid tile elements, keep all threads participating in barriers, and guard stores.	Safe edge tiles.	Returning some threads before <code>__syncthreads</code> .
7. Why can optimized GEMM be compute-bound?	Tiling greatly raises arithmetic intensity by reusing each loaded value.	Roofline change.	Treating all GPU kernels as bandwidth-bound.
8. What are bank conflicts?	Shared-memory lanes access addresses mapping to the same bank in a conflicting pattern, serializing service.	On-chip layout.	Confusing them with global coalescing.
9. Why accumulate in FP32?	Reduces error/range problems for many low-precision products.	Mixed precision.	Assuming accumulation order/precision does not matter.

Question	Answer	Expected	Common mistake
10. Build or use cuBLAS?	Use a tuned library in production unless requirements are unusual; implement to learn or fuse specialized work.	Engineering judgment.	Claiming a classroom kernel beats vendor GEMM generally.

7. Deep-Dive Questions

1. **How does tile size affect performance?** Larger tiles increase reuse but consume more shared memory and registers, may lower occupancy, and must fit hardware limits. Benchmark architecture- and shape-specific choices.
2. **How do tensor cores change the mapping?** They perform small matrix multiply-accumulate fragments per warp using supported data formats. Threads cooperatively load fragments; layout, alignment, and accumulation type become crucial.
3. **What is split-K?** Multiple blocks process different ranges of K for the same output tile, then combine partial sums. It increases parallelism for small M, N but needs another reduction/atomic path.
4. **Why is floating-point GEMM not bitwise invariant?** Addition is non-associative; tiling, FMA, and parallel reduction change operation order. Reproducibility may require constrained algorithms at a performance cost.
5. **What does $C = \alpha * AB + \beta * C$ change?** The epilogue reads old C when $\beta \neq 0$, scales the accumulator, and may fuse bias/activation, saving separate passes.

8. Comparison Tables

Naive GEMM	Tiled GEMM
Repeated global loads	Cooperative loads with reuse
Simple, useful reference	More barriers and edge logic
Usually bandwidth-limited	Can approach compute limit
Easy correctness baseline	Sensitive to tile/resource choices
Matrix addition	Matrix multiplication
Element-wise	Row-column reduction
Equal shapes	Matching inner dimensions
$O(MN)$	$O(MNK)$
No shared reuse needed	Reuse/tile is critical
Shared memory	Registers
Shared by block	Private to thread
Good for cooperative tiles	Fastest for per-thread accumulators
Requires barriers	Too many registers reduce occupancy

9. Common Mistakes

- Getting shape rules or row-major indexing wrong.
- Loading tiles but failing to reuse them enough to justify overhead.
- Placing a conditional return before a block-wide barrier.
- Omitting the barrier before overwriting shared tiles.
- Ignoring shared-memory bank conflicts and register pressure.
- Comparing an educational kernel unfairly with a tuned library.
- Assuming maximum occupancy equals maximum GEMM performance.

10. Edge Cases / Special Cases

- M , N , or K zero: API semantics determine whether output is untouched, zeroed, or scaled by β .
- Partial boundary tiles: invalid inputs contribute zero; invalid outputs are not stored.
- K very large: accumulator error and range deserve attention.
- Highly skinny matrices may need GEMV/specialized kernels instead of general GEMM.
- Aliasing C with A or B is generally unsafe because outputs overwrite values still needed.
- Transposed operands should be represented through layout/leading dimensions or specialized kernels, not necessarily explicit transpose copies.

11. How to Explain in Interview

“Matrix multiplication computes each output as a row-column dot product. A naive GPU gives one output to each thread, but neighboring outputs repeatedly load the same inputs. A tiled kernel lets a block cooperatively load A and B tiles into shared memory, synchronize, reuse them for many FMAs, and iterate over K . That raises arithmetic intensity; edge loads are zero-filled and all threads reach barriers. In production I would use a tuned BLAS library unless specialization or fusion justifies custom code.”

12. Quick Revision Notes

- $(M \times K) (K \times N) \rightarrow M \times N$; work is $O(MNK)$.
- One output = dot product = reduction over K .
- Tiling trades shared memory/registers/barriers for global-memory reuse.
- Guard boundary loads/stores; never let only part of a block reach a barrier.
- Mixed precision commonly uses low-precision inputs and FP32 accumulation.
- Trap: high occupancy is a means, not the goal.

13. Practice Tasks

1. Write a CPU reference and naive GPU GEMM; test nonsquare shapes.
2. Implement a tiled kernel with boundary zero-fill and compare correctness.
3. Count approximate global loads in naive versus tiled versions.
4. Benchmark several tile sizes and inspect registers, occupancy, and bandwidth.
5. Compare with cuBLAS and explain the performance gap.
6. Extend the epilogue to $\alpha * AB + \beta * C$ and optional ReLU.

14. Final Cheat Sheet

Item	Recall
Core	$C[i,j] = \sum_k A[i,k]B[k,j]$
Why	Dominant compute primitive in ML/science/graphics
Most asked	Shape, tiling, coalescing, barriers, edges, precision
Compare	Naive vs tiled vs vendor library; shared memory vs registers
One line	Tile and reuse A/B on chip so many FMAs are performed per global byte.

Image Operations

1. Overview

Definition. GPU image operations transform pixels. **Point operations** use only the current pixel (brightness, threshold, color conversion); **neighborhood operations** use nearby pixels (blur, sharpen, edge detection, morphology); **geometric operations** sample another coordinate (resize, rotate, warp).

Images provide millions of similar data items, making them natural GPU workloads. Cameras, browsers, games, medical imaging, computer vision, and video encoders use these kernels. Interviewers ask them to test 2D indexing, channel/layout choices, memory coalescing, border policy, convolution tiling, texture/sampling hardware, precision, and the difference between mathematically correct and visually correct output.

2. Core Idea

Think of an image as a grid of colored tiles. A brightness worker changes only its tile. A blur worker must borrow neighboring tiles. A resize worker asks, "Which source coordinate corresponds to my output tile?"

Example 3×3 box blur for an interior pixel:

```
10 20 30
20 40 60  -> center output = sum(all 9)/9 = 40
30 60 90
```

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
if (x < width && y < height) {
    // point operation example
    out[y * stride + x] = clamp(in[y * stride + x] * gain, 0, 255);
}
```

For a neighborhood filter, a block often loads an image tile plus a **halo** into shared memory. Threads reuse overlapping neighborhoods, synchronize, compute, and store. Geometric kernels usually assign one thread per destination pixel (inverse mapping), sample the source, and avoid holes.

3. Important Subtopics

Subtopic	Meaning / why it matters	Example	Interview angle
Pixel representation	Interleaved RGBRGB versus planar channels; 8-bit, half, float.	uchar4 RGBA.	Layout drives addressing/coalescing.
Stride/pitch	Bytes between row starts may exceed width*channels .	Camera frames with padding.	Never assume tightly packed rows.
Point kernels	One input pixel per output; no neighborhood reuse.	Gamma/threshold.	Usually bandwidth-bound and fusible.
Convolution/stencil	Weighted neighborhood sum.	Sobel, Gaussian blur.	Halo, reuse, separability.
Border policy	Define out-of-range samples.	Clamp, reflect, wrap, constant, valid-only.	Correctness requirement, not cleanup.
Sampling	Nearest, bilinear, bicubic trade quality for work.	Resize.	Coordinate convention and interpolation.
Shared-memory tiles	Load tile plus halo once for reuse.	16×16 output with radius 1 loads 18×18.	Barriers and bank/layout issues.
Texture/read-only path	Hardware caching/address modes/interpolation may suit spatial sampling.	Bilinear warp.	Use when semantics and platform support match.
Separable filters	2D kernel factors into horizontal and vertical 1D passes.	Gaussian.	$O(r^2)$ to $O(r)$ samples per pixel, but extra pass.

4. Real-World Example

A browser decodes a photo, converts color space, resizes it for layout, and composites it with opacity. Point steps can be fused; resize uses inverse coordinate mapping with bilinear interpolation; compositing respects alpha semantics and color space. The image may have padded rows, and edge sampling must be specified. The GPU wins because the frame is large and often already resident for display.

5. Diagrams / Mental Models

```

Shared-memory stencil tile (radius 1)
+-----+
| halo halo halo |
| halo OUTPUT halo |   load 18x18 to compute 16x16
| halo halo halo |
+-----+

```

Resize by inverse mapping:
 destination pixel (xd,yd) -> source coordinate (xs,ys) -> sample neighbors

Operation class	Dependencies	Typical optimization
Point	Same pixel	Coalescing + fusion
Neighborhood	Nearby pixels	Shared tile/halo, separability
Geometric	Sampled coordinates	Texture/cache, interpolation
Global	Many/all pixels	Reduction/histogram/scan stages

6. Common Interview Questions

Question	Answer	Expected points	Common mistake
1. How map threads to an image?	Usually 2D grid; x follows columns and y rows; address with byte/element stride.	Bounds and layout.	Using width as stride unconditionally.
2. Why use a halo?	Boundary threads load neighboring pixels required by the output tile.	Overlapping neighborhood reuse.	Loading only output-sized data.
3. How handle image borders?	Choose clamp/reflect/wrap/constant/valid semantics explicitly.	Defined output.	Reading out of bounds or silently shrinking.
4. Why inverse-map a resize?	Every destination pixel gets exactly one source sample; forward mapping can leave holes/collisions.	Output ownership.	One thread per source for arbitrary warp.
5. Nearest vs bilinear?	Nearest selects one sample; bilinear blends four, smoother but costlier.	Quality/work tradeoff.	Averaging without correct fractional weights.
6. What is separable convolution?	A 2D filter represented as horizontal then vertical 1D filters.	Reduced operations.	Assuming every kernel is separable.
7. When shared memory?	When overlapping neighborhoods create enough reuse to repay halo loads/barriers.	Reuse criterion.	Using it for point transforms.
8. Why <code>uchar4</code> ?	Matches RGBA packing and may enable aligned vector access; arithmetic still needs widening/clamping.	Representation/alignment.	Adding 8-bit values and allowing wraparound.

Question	Answer	Expected points	Common mistake
9. Is image processing always bandwidth-bound?	Point kernels often are; large convolutions can be compute-heavy.	Operation-dependent roofline.	One classification for all filters.
10. Why can GPU output differ?	Precision, FMA/order, rounding, interpolation convention, and color-space assumptions.	Numerical/semantic details.	Calling every difference a race.

7. Deep-Dive Questions

- How do you load a halo efficiently?** Let all block threads cooperatively cover the larger tile with looped loads, apply border policy during loading, then synchronize once. Avoid divergent bespoke corner code when a simple linear cooperative load works.
- When are two separable passes slower?** For tiny kernels or small images, the intermediate global write/read and second launch can outweigh fewer arithmetic operations; fusion or direct convolution may win.
- What is the alpha-compositing trap?** Straight and premultiplied alpha use different formulas. Interpolating straight RGB near transparent pixels can create fringes; systems often use premultiplied values.
- Why does color space matter?** Averaging sRGB-encoded values is not the same as averaging linear light. High-quality filtering converts or uses a linear representation.
- How can occupancy fall in a convolution?** Large shared tiles and many per-thread accumulators consume shared memory/registers, limiting resident blocks; tune tile shape and channel handling.

8. Comparison Tables

Point operation		Neighborhood operation	
One pixel/input coordinate		Window around coordinate	
No halo/barrier		Tile + halo often beneficial	
Usually bandwidth-bound		Reuse may raise arithmetic intensity	
Easy fusion		Border and radius complicate fusion	
Nearest	Bilinear	Bicubic	
1 sample, blocky	4 samples, smooth	Usually 16 samples, sharper/smooth	
Cheapest	Common default	More compute and possible ringing	
Interleaved		Planar	
Channels adjacent per pixel		Each channel is a separate plane	
Convenient for display/RGBA		Convenient for channel-wise compute/ML	

Interleaved**Planar**

Packed vector loads possible

Same-channel lanes remain contiguous

9. Common Mistakes

- Treating stride as `width*channels` for every image.
- Swapping x/y or mixing byte stride with element indexing.
- Leaving border semantics implicit.
- Returning some threads before a block-wide synchronization.
- Forgetting halo pixels or redundantly loading them per output.
- Performing arithmetic in `uint8_t` without widening and saturation.
- Resizing by forward mapping and producing holes.
- Ignoring color space and alpha representation.

10. Edge Cases / Special Cases

- Empty images, one-pixel dimensions, and filters larger than the image.
- Odd row pitches, regions of interest, and negative/top-down versus bottom-up row conventions.
- Grayscale, RGB, RGBA, and unusual channel counts/layouts.
- NaN/Inf in floating-point images and saturation for integer output.
- Overlapping input/output is safe for point kernels but usually unsafe for neighborhoods.
- Coordinate conventions (pixel centers versus corners) can shift resized output by half a pixel.

11. How to Explain in Interview

"I classify the image kernel first: point, neighborhood, or geometric. I map a 2D grid with x along contiguous pixels and use the true row stride. Point operations are independent and fusible. For filters, a block can cooperatively load a tile plus halo into shared memory and reuse it after a barrier. For resize or warp, one thread owns each destination pixel and inverse-maps to the source. I define borders, interpolation, channel layout, precision, and alpha/color-space semantics explicitly."

12. Quick Revision Notes

- Address with `(x,y,channel)` plus real pitch/stride.
- Point: same pixel; stencil: neighborhood; warp: sampled coordinate.
- Tile + halo only when neighborhood reuse justifies it.
- Inverse mapping prevents holes; interpolation convention matters.
- Traps: borders, `uint8` overflow, half-pixel offsets, alpha/color space.

13. Practice Tasks

1. Implement grayscale conversion and thresholding; fuse them.
2. Write naive and tiled 3×3 blur with four border modes.
3. Implement Sobel edge detection and verify a tiny hand-computed image.
4. Implement nearest and bilinear resize using inverse mapping.
5. Compare direct 2D Gaussian blur with two separable passes.
6. Support padded row stride and an ROI not starting at `(0,0)`.

14. Final Cheat Sheet

Item	Recall
Core	Parallel pixel transforms: point, neighborhood, geometric, or global
Why	Vision, browser, media, camera, medical, graphics workloads
Most asked	2D addressing, pitch, halo, borders, interpolation, layout
Compare	Point vs stencil; nearest vs bilinear; interleaved vs planar
One line	Map outputs to threads, make rows coalesced, and define sampling/borders before optimizing reuse.

Reduction

1. Overview

Definition. A reduction combines many values into fewer values using an associative operator: sum, product, minimum, maximum, logical AND/OR, or a custom associative combine. A full reduction produces one value; segmented/batched reductions produce one value per group.

Reductions power statistics, loss computation, norms, database aggregation, scientific solvers, and ML normalization. Unlike map kernels, outputs depend on many inputs, so threads must cooperate. Interviewers ask reductions to test associativity, tree algorithms, synchronization scope, warp primitives, shared memory, atomics, numerical accuracy, and multi-kernel decomposition.

2. Core Idea

Passing every number to one worker is serial. Instead, pair workers combine values in a tournament tree:

```
[3 1 7 2 4 6 5 8]
[4] [9] [10] [13]
[13] [23]
[36]
```

Parallel work stays $O(n)$, while ideal depth becomes $O(\log n)$. On a GPU, each block reduces a chunk to one partial result using registers, warp shuffles, and/or shared memory. A later kernel reduces block partials. Atomics can combine partials when their contention is acceptable.

3. Important Subtopics

Subtopic	Meaning / why it matters	Example	Interview angle
Associativity	Grouping must not change mathematical meaning.	$(a+b)+c=a+(b+c)$ mathematically.	Floating point is only approximately

Subtopic	Meaning / why it matters	Example	Interview angle
			associative.
Identity	Neutral value initializes inactive lanes.	Sum 0, min $+\infty$, max $-\infty$.	Wrong identity breaks negative/empty cases.
Tree reduction	Halve active values per step.	$256 \rightarrow 128 \rightarrow \dots \rightarrow 1$.	$O(\log n)$ depth and sync.
Warp shuffle	Lanes exchange register values without shared memory.	<code>shuffle_down</code> sum.	Valid masks and warp scope.
Block reduction	Warps reduce, their leaders write partials, one warp finishes.	One output/block.	Block barrier between stages.
Grid reduction	Blocks cannot normally barrier globally inside an ordinary kernel.	Second kernel or cooperative launch.	Kernel boundary as global synchronization.
Atomics	Safely update shared result but serialize conflicting updates.	One atomic per block.	Better than one atomic per element.
Accuracy	Order changes rounding; compensation/pairwise trees improve error.	FP32 sum.	Determinism versus speed.

4. Real-World Example

A softmax layer needs the maximum of each row, then the sum of exponentials. Each row uses reductions, often with one block or warp per row. Subtracting the maximum prevents overflow. Production kernels may combine maximum, exponentiation, sum, and normalization while carefully synchronizing, showing that reduction is both an algorithm and a building block inside larger kernels.

5. Diagrams / Mental Models

```

global input
| chunks
[block 0] [block 1] [block 2] [block 3]
  p0      p1      p2      p3
   \      |      |      /
    second reduction -> result

```

Strategy	Global atomics	Launches	Best fit
One atomic/input	n	1	Rarely; tiny input/low contention
One atomic/block	blocks	1	Simple when atomic throughput is sufficient
Hierarchical partials	0 or final few	2+	General scalable reduction

6. Common Interview Questions

Question	Answer	Expected	Common mistake
1. What is a reduction?	Combine a collection with an associative operator into one/fewer values.	Operator and grouping.	Describing only sum.
2. Work and depth?	$O(n)$ total work, $O(\log n)$ ideal tree depth.	Work-depth distinction.	Saying total work is $O(\log n)$.
3. Why require associativity?	Parallel trees regroup operations nondeterministically/differently.	Correct regrouping.	Requiring commutativity but missing associativity.
4. Why multiple kernels?	Ordinary blocks lack a grid-wide barrier; launch boundaries synchronize stages.	Synchronization scope.	Calling <code>__syncthreads()</code> grid-wide.
5. Why not one atomic/thread?	Heavy contention serializes access and wastes throughput.	Hierarchical combine.	Saying atomics are always incorrect.
6. What identity for max?	Lowest possible value/negative infinity, not zero.	Negative inputs.	Initializing max to zero.
7. Are FP sums deterministic?	Not generally; different grouping/scheduling changes rounding.	Non-associativity.	Assuming atomic order is fixed.
8. What does a warp shuffle do?	Moves register values between lanes for warp-local combine.	Lower shared-memory/sync overhead.	Using it across warps.
9. Handle arbitrary n?	Bounds-load identity; grid-stride through input; reduce valid values.	Safe tail.	Reading the padded tail.
10. Sum overflow?	Use a wider accumulator or explicit modular/saturating semantics.	Accumulator type.	Widening only after overflow occurred.

7. Deep-Dive Questions

1. **How would you make a reduction deterministic?** Fix partitioning and tree order, avoid unordered atomics, and use reproducible algorithms; expect performance cost and still specify platform/precision.
2. **What is Kahan summation on a GPU?** Each worker can keep a compensation term for its serial chunk, then combine partials carefully. It improves accuracy but increases instructions/registers and does not make arbitrary merging trivial.
3. **When does one block suffice?** When one block can cover or grid-stride through the input efficiently and enough parallelism remains—often for small reductions or one block per segment/row.
4. **How reduce a custom structure?** Define an associative combine and identity, e.g., `(sum, count)` for a mean; watch alignment, register pressure, and proof of associativity.
5. **What is argmax?** Reduce pairs `(value, index)` using a comparison plus a defined tie-breaker; NaN policy and stable lowest-index behavior must be explicit.

8. Comparison Tables

Reduction	Scan
One/few aggregate outputs	Prefix result for every position
Tree collapses data	Up-sweep/down-sweep or staged prefix propagation
Sum gives total only	Last inclusive sum equals total
Less output traffic	More output and coordination
Shared-memory tree	Warp shuffle
Works across whole block	Directly within a warp
Needs barriers between cross-warp stages	Warp-level synchronization/mask rules
Flexible types/layouts	Best for register-sized values

9. Common Mistakes

- Assuming `__syncthreads()` synchronizes different blocks.
- Using a non-associative operator without defining order semantics.
- Choosing zero as min/max identity.
- Diverging around a barrier.
- Doing one global atomic per element without measuring contention.
- Ignoring floating-point order, overflow, NaNs, and argmax tie-breaking.

10. Edge Cases / Special Cases

- Empty input needs an API decision: return identity, error, or optional value.
- `n=1` should return the element without special synchronization problems.
- Non-power-of-two sizes require guarded loads/identity padding.
- NaN min/max semantics differ between “propagate NaN” and “ignore NaN.”
- Signed integer overflow and very large counts require wider types.
- Segments may be empty or highly imbalanced.

11. How to Explain in Interview

“A reduction combines inputs with an associative operator. I use a hierarchical tree: each thread accumulates a local chunk, each warp reduces registers, a block combines warp partials, and another kernel or a small number of atomics combines block results. The tree has linear work and logarithmic depth. I define the identity, tail handling, NaN/overflow policy, and acknowledge that floating-point regrouping affects reproducibility.”

12. Quick Revision Notes

- Associative operator + identity; commutativity is useful but not always mandatory if order is preserved.
- $O(n)$ work, $O(\log n)$ tree depth.
- Warp $\rightarrow$ block $\rightarrow$ grid hierarchy mirrors hardware synchronization scopes.
- Kernel launch provides global stage separation.

- Traps: wrong identity, block-only barrier, floating-point nondeterminism.

13. Practice Tasks

1. Implement CPU and GPU sum for non-power-of-two sizes.
2. Build max and argmax with negative values and deterministic ties.
3. Compare one-atomic-per-element, one-atomic-per-block, and two-pass versions.
4. Measure error against a double-precision reference for adversarial FP32 input.
5. Implement one reduction per matrix row for softmax preparation.

14. Final Cheat Sheet

Item	Recall
Core	Associatively combine many values into one/few results
Why	Sums, extrema, norms, database aggregates, ML statistics
Most asked	Tree depth, barriers, atomics, identity, FP accuracy
Compare	Reduction vs scan; atomics vs hierarchical partials
One line	Reduce locally and hierarchically, matching warp/block/grid synchronization scopes.

Prefix Sum / Scan

1. Overview

Definition. Scan computes all prefixes of a sequence. Inclusive sum scan of $[3, 1, 4, 2]$ is $[3, 4, 8, 10]$; exclusive scan is $[0, 3, 4, 8]$. Scan generalizes from addition to any associative operator with an identity.

Scan is used for stream compaction, radix sort, allocation offsets, sparse structures, parsing, cumulative distributions, and graph frontiers. It turns per-element flags/counts into stable output positions. Interviewers ask it because it requires deeper parallel dependency reasoning than reduction: every prefix is needed, work efficiency matters, and block-level scans must be composed globally.

2. Core Idea

A serial running total depends on the previous result. A parallel scan reorganizes dependencies into a tree. In the work-efficient Blelloch exclusive scan, an **up-sweep** builds partial sums, the root is replaced by the identity, and a **down-sweep** distributes prefix values.

```
input:      [3 1 4 2]
inclusive: [3 4 8 10]
exclusive: [0 3 4 8]
```

For a large array: scan each block, store each block total, scan the block totals, then add the scanned block offset to every element in that block. This is the central composition pattern.

3. Important Subtopics

Subtopic	Meaning / why it matters	Example	Interview angle
Inclusive/exclusive	Includes current input or stops before it.	Above example.	Convert using shift/add.
Hillis–Steele	At distance 1,2,4... each element adds a predecessor; simple but $O(n \log n)$ work.	Warp scan.	Depth-efficient, not work-efficient.
Blelloch	Up-sweep/down-sweep; $O(n)$ work, $O(\log n)$ depth.	Block exclusive scan.	Explain root reset and both phases.
Warp scan	Shuffle-based lane prefixes.	32 values.	Lane masks and offsets.
Hierarchical scan	Local scans + scan block sums + uniform add.	Large arrays.	Global composition.
Stream compaction	Scan 0/1 keep flags to produce stable positions.	Filter positives.	Canonical application.
Segmented scan	Prefix resets at segment boundaries.	Per-row ragged data.	Carry flag/value pairs.
In-place safety	Algorithms need careful staged reads/writes and barriers.	Shared-memory scan.	Avoid read-after-write races.

4. Real-World Example

To remove invalid records on a GPU, each thread writes `flag[i]=1` if its record is valid. An exclusive scan gives `position[i]`, the number of valid records before `i`. Threads with flag 1 write to `output[position[i]]`. The last position plus last flag gives the output size. This creates stable, collision-free destinations without a global atomic per item.

5. Diagrams / Mental Models

```
records: [A B C D E]
keep:    [1 0 1 1 0]
scan:    [0 1 1 2 3] (exclusive)
write:   A  C D      -> output [A C D]

large scan:
local block scans -> block totals -> scan totals -> add block offsets
```

Algorithm	Work	Depth	Typical use
Serial	$O(n)$	$O(n)$	CPU/small input

Algorithm	Work	Depth	Typical use
Hillis–Steele	$O(n \log n)$	$O(\log n)$	Simple/warp-sized
Blelloch	$O(n)$	$O(\log n)$	Work-efficient block scan

6. Common Interview Questions

Question	Answer	Expected	Common mistake
1. Inclusive vs exclusive?	Inclusive includes <code>x[i]</code> ; exclusive returns the prefix before <code>i</code> and starts with identity.	Exact example.	Off-by-one definitions.
2. Scan vs reduction?	Reduction returns total; scan returns every prefix.	Output/dependency difference.	Calling scan a cumulative reduction only without parallel distinction.
3. Why is naive parallel scan inefficient?	Hillis–Steele performs n work over $\log$ stages: $O(n \log n)$.	Work efficiency.	Claiming depth equals work.
4. Blelloch phases?	Up-sweep builds sums; root becomes identity; down-sweep propagates prefixes.	Tree understanding.	Forgetting root reset.
5. Scan arrays larger than a block?	Scan blocks, scan block totals, add scanned offsets.	Hierarchical composition.	Assuming block barrier is global.
6. Why exclusive scan for compaction?	It gives the exact number of kept items before each item—its output index.	Address allocation.	Using inclusive result directly without subtracting/adjusting.
7. Complexity?	Work-efficient scan has $O(n)$ work and $O(\log n)$ ideal depth.	Both metrics.	Saying $O(\log n)$ total.
8. Can scan use max?	Yes, for associative operators with an identity.	Generality.	Restricting it to sums.
9. Handle non-power-of-two n ?	Pad inactive slots with identity or use guarded generalized algorithms.	Correct tails.	Reading/writing padding as real data.
10. What is segmented scan?	Prefix operation resets at marked segment starts.	Ragged/grouped data.	Running one global prefix across boundaries.

7. Deep-Dive Questions

1. **How do you convert an exclusive sum scan to inclusive?** Add the original element to each exclusive result. Conversely, shift inclusive output right and place identity at index zero.
2. **Can a scan be completed in one kernel?** Special cooperative-grid or decoupled-look-back algorithms can, under platform constraints. The portable explanation is hierarchical multi-kernel scan.
3. **How do bank conflicts arise?** Tree strides can map many shared addresses to the same banks. Padding/index remapping or shuffle-based warp stages reduces conflicts.
4. **How does segmented scan combine pairs?** Carry (`value`, `headFlag`); when the right operand begins a segment, discard the left accumulated value, otherwise combine. The pair operator must be associative.
5. **Is floating-point scan reproducible?** A fixed tree can be deterministic for a fixed configuration, but differs numerically from serial left-to-right addition due to regrouping.

8. Comparison Tables

Inclusive	Exclusive
$y[i]=x[0]\oplus\ldots\oplus x[i]$	$y[i]=identity\oplus x[0]\ldots x[i-1]$
Final element is total	Final total needs last output combined with last input
Natural cumulative metric	Natural offset/allocation primitive
Hillis-Steele	Blelloch
Simple iterative offsets	Up-sweep and down-sweep tree
$O(n \log n)$ work	$O(n)$ work
Useful at small/warp scale	Common conceptual block scan

9. Common Mistakes

- Mixing inclusive and exclusive semantics.
- Reporting logarithmic depth as logarithmic total work.
- Forgetting to scan block totals before adding offsets.
- Using the unscanned block total as every block's offset.
- Omitting a stage barrier in shared memory.
- Assuming power-of-two length or addition-only behavior.
- Getting compaction's final output count wrong.

10. Edge Cases / Special Cases

- Empty input produces empty output; its aggregate is the identity if requested.
- One element: inclusive is that element; exclusive is identity.
- Non-power-of-two sizes and final partial blocks use identity padding.
- Integer prefix sums may overflow even when each element is small; positions often need 64-bit types.
- Segments can be empty or begin at index zero; head-flag convention must be clear.
- In-place global scan is possible with correct staging but careless updates create races.

11. How to Explain in Interview

“Scan returns every prefix, either inclusive or exclusive. A work-efficient parallel scan uses a tree with linear work and logarithmic depth. For arrays larger than a block, each block scans locally and emits its total, those totals are scanned, and each block receives a uniform prefix offset. Exclusive scan is especially useful because scanning 0/1 flags converts them into stable output indices for compaction.”

12. Quick Revision Notes

- Inclusive includes current item; exclusive starts with identity.
- Work-efficient: $O(n)$ work, $O(\log n)$ depth.
- Large scan = local scans + scanned block totals + uniform add.
- Scan generalizes to associative operators.
- Trap: Hillis–Steele is parallel but not work-efficient.

13. Practice Tasks

1. Hand-trace Blelloch scan for 8 elements.
2. Implement warp inclusive and block exclusive sum scans.
3. Extend to arbitrary sizes with hierarchical block totals.
4. Build stable stream compaction from flags + exclusive scan.
5. Implement segmented scan using head flags.
6. Compare results and error with serial FP32 scanning.

14. Final Cheat Sheet

Item	Recall
Core	Compute every associative prefix, inclusive or exclusive
Why	Compaction, sorting, offsets, sparse data, graph frontiers
Most asked	Blelloch phases, work/depth, hierarchy, compaction
Compare	Inclusive vs exclusive; scan vs reduction; Hillis–Steele vs Blelloch
One line	Scan blocks locally, scan their totals, then add each block's prefix offset.

Histogram

1. Overview

Definition. A histogram counts how many input items fall into each bin. For byte values, `hist[v]` is the number of pixels/elements equal to `v`; general histograms map ranges or categories to bins.

Histograms are used in image equalization, database group-by/count, telemetry, feature extraction, probability estimation, radix sort, and anomaly detection. They are parallel but not embarrassingly parallel because many threads may update the same bin. Interviewers ask them to test atomic correctness, contention, privatization, shared-memory capacity, skewed distributions, bin mapping, overflow, and multi-stage merging.

2. Core Idea

Imagine shoppers dropping tokens into labeled buckets. They can classify items independently, but simultaneous drops into the same bucket must not lose counts. A naive GPU uses `atomicAdd(&hist[bin(x[i])],1)`. It is correct but hot bins serialize.

The common optimization is **privatization**: each block (or warp/thread where feasible) builds a private histogram, usually in shared memory, then merges private counts into the global histogram. This spreads contention during the high-volume input pass and reduces global atomics.

```
input: [2 0 2 1 2 0]
bins:  0 -> 2, 1 -> 1, 2 -> 3

block private histograms -> global merge -> final histogram
```

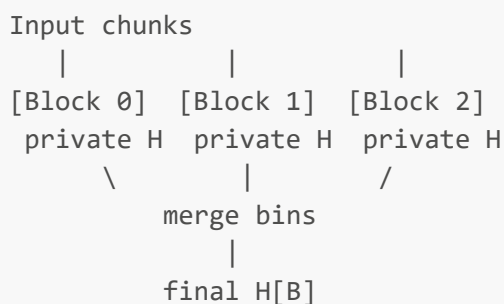
3. Important Subtopics

Subtopic	Meaning / why it matters	Example	Interview angle
Bin mapping	Convert value to valid bin index.	<code>floor((x-min)/width)</code> .	Endpoints, clamp/reject policy.
Atomic update	Read-modify-write without lost increments.	<code>atomicAdd</code> .	Correctness versus contention.
Privatization	Replicate bins per block/warp/thread, then merge.	Shared histogram per block.	Memory cost vs fewer global conflicts.
Initialization	Every private/global bin must start at zero.	Cooperative clear.	Barrier after shared clear.
Merge	Combine private histograms into final bins.	One atomic per block/bin or second kernel.	Scalable final stage.
Skew	Nonuniform inputs create hot bins.	All pixels zero.	Worst-case benchmark.
Bin count	Determines shared-memory fit and initialization/merge cost.	256 byte bins vs millions.	Strategy changes with B .
Counter width	Maximum count may exceed 32-bit.	Billions of records.	Overflow and atomic support.

4. Real-World Example

Image histogram equalization first computes 256 luminance counts, scans them to form a cumulative distribution, then maps pixels through that distribution. A shared-memory histogram per block handles pixels; block histograms are merged; a scan produces the CDF. This pipeline connects histogram, reduction-like merging, scan, and a final point image operation.

5. Diagrams / Mental Models



Design	Storage	Contention	Good when
Global atomic/input	B global counters	Potentially high	Small/simple or atomics are sufficient
Block-private	$\text{blocks} \times B$ logically/on chip	Local, then merge	B fits shared memory
Warp-private	$\text{warps} \times B$	Lower local contention, more storage	Small B , severe skew

6. Common Interview Questions

Question	Answer	Expected	Common mistake
1. Why need atomics?	Increment is read-modify-write; concurrent non-atomic increments lose updates.	Race explanation.	Assuming aligned writes are automatically additive.
2. Why can naive atomics be slow?	Threads targeting one/few counters contend and serialize.	Input distribution matters.	Saying atomics always serialize all bins.
3. What is privatization?	Make independent partial histograms and merge later.	Trade memory/merge for contention.	Private histogram per thread regardless of size.
4. Why shared memory?	Faster on-chip block-private updates and fewer global atomics.	Scope and fit.	Forgetting shared atomics/contention still exist.
5. What barriers are needed?	After cooperative zeroing, and before any thread begins merging after updates.	Phase safety.	Clearing and updating concurrently.
6. How handle many bins?	Tile bins, use global partials/sort-based methods, or sparse representation depending on data.	Shared-memory capacity.	Allocating impossible per-block shared arrays.
7. Worst-case input?	Every item maps to the same bin, maximizing contention.	Skew robustness.	Benchmarking only uniform random data.

Question	Answer	Expected	Common mistake
8. Complexity?	$O(n+B)$ conceptual work including initialization; parallel time depends heavily on contention/merge.	Complexity plus hardware behavior.	Claiming atomics make it $O(1)$.
9. How map float values?	Define range and bin width, then floor/map with explicit underflow/overflow and max-endpoint policy.	Boundary correctness.	Letting <code>value==max</code> produce bin <code>B</code> .
10. Counter type?	Choose width for maximum possible count; often 64-bit for huge data, considering atomic availability/cost.	Overflow.	Using 8/16-bit counters because inputs are bytes.

7. Deep-Dive Questions

- When is sorting better than atomics?** For very large/sparse bin spaces or extreme contention, sort keys then run-length encode/reduce. Sorting costs more work but turns random updates into grouped processing.
- How mitigate shared-memory bank conflicts?** Replicate/pad hot bins or use warp-private subhistograms so simultaneous updates spread across banks/addresses, then reduce replicas.
- How produce a weighted histogram?** Atomically add a weight rather than one. Floating-point atomic ordering makes results nondeterministic; precision and overflow/NaN policy must be specified.
- What if bins are dynamic strings/categories?** First encode categories (hash/table/dictionary) or sort/group keys; a dense fixed-bin array only suits a bounded known key space.
- Can you avoid initializing the whole histogram?** Sparse touched-bin tracking or generation tags can help when `B` is huge and few bins occur, but add metadata and contention; use only when measurement supports it.

8. Comparison Tables

Global atomic	Privatized histogram	
Minimal code/storage	Extra partial storage and merge	
Correct and often adequate	Reduces global contention	
Sensitive to skew/hot bins	Still has local contention	
Best baseline	Best common optimization when bins fit	
Histogram	Reduction	Scan
Many counters selected by data	One/few combined values	Every prefix
Scatter/update contention	Hierarchical combine	Prefix propagation
Output size <code>B</code>	Usually 1/group	Usually <code>n</code>

9. Common Mistakes

- Incrementing bins without atomics or ownership partitioning.
- Forgetting to zero global/private counters.
- Missing barriers between clear, update, and merge phases.
- Assuming uniform input and ignoring hot-bin contention.
- Mapping the upper endpoint to index B .
- Using counters too narrow for n .
- Creating so many private bins that occupancy collapses or shared memory overflows.

10. Edge Cases / Special Cases

- Empty input should leave all bins zero.
- $B=1$ is maximal contention and a useful stress test.
- Values below/above range require discard, clamp, or overflow bins.
- `value==max` needs a specified inclusive endpoint rule.
- NaN cannot be safely converted to an ordinary bin without policy.
- Huge B may make $O(B)$ initialization dominate $O(n)$ counting.
- Final counters may exceed 32 bits; weighted counters may need floating-point accuracy handling.

11. How to Explain in Interview

"A histogram maps each input to a bin and increments that counter. The challenge is contention: non-atomic increments lose updates, while global atomics can serialize on hot bins. I start with a correct atomic baseline, then, when the bin count fits, give each block or warp a private shared-memory histogram, synchronize its initialization and updates, and merge partial bins globally. I test uniform and worst-case skew and define range endpoints, outliers, NaNs, and counter width."

12. Quick Revision Notes

- Classification is parallel; counter updates conflict.
- Correct baseline: global atomic increment.
- Common optimization: privatize $\rightarrow$ local atomic updates $\rightarrow$ merge.
- Strategy depends on n , bin count B , and skew.
- Traps: zeroing/barriers, endpoint mapping, overflow, all-values-one-bin.

13. Practice Tasks

1. Implement a 256-bin byte histogram with global atomics.
2. Add block-private shared histograms and merge them.
3. Benchmark uniform, single-bin, and Zipf-like inputs.
4. Implement float binning with explicit underflow/overflow bins.
5. Use histogram + exclusive scan to build histogram equalization or counting-sort offsets.
6. Verify that all bin counts sum to the number of accepted inputs.

14. Final Cheat Sheet

Item	Recall
Core	Map each item to a bin and count items per bin

Item	Recall
Why	Images, analytics, telemetry, sorting, probability features
Most asked	Atomics, contention, privatization, skew, bin mapping
Compare	Global atomics vs block/warp-private histograms
One line	Make increments correct with atomics, then reduce hot-bin contention through measured privatization and merging.

Cross-Kernel Placement Summary

Kernel	Output dependency	Main bottleneck	Primary GPU technique
Vector addition/scaling	Independent per element	Global bandwidth / launch	Coalescing, fusion
Matrix addition	Independent per cell	Global bandwidth	Layout-aware 1D/2D mapping
Matrix multiplication	Reduction per output, heavy reuse	Compute/on-chip resources when tuned	Tiling, shared memory, registers
Image operations	Point, neighborhood, or sampled	Depends on filter	2D mapping, halo, sampling
Reduction	Many-to-one	Synchronization/traffic/atomics	Hierarchical tree
Scan	Prefix dependencies	Coordination and traffic	Local scan + block offsets
Histogram	Conflicting bin updates	Atomic contention/skew	Privatization + merge